程序中的并发
多线程程序设计

主讲老师： 申雪萍

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

主要内容

- **进程的概念**
- **线程的概念**
- 线程的调度
- 创建和启动线程
  - Thread线程类、
  - Runnable接口
- 多个线程的同步
- 线程之间的通信

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

程序（静态蓝本）和进程（动态执行）

- 程序（program）：是一段静态的代码，是对数据描述与操作的代码的集合，它是应用程序执行的蓝本。
- 进程（process）是**程序的一次执行过程**，是系统运行程序的基本单位。程序是静态的，进程是动态的。系统运行一个程序即是一个进程从创建、运行到消亡的过程。
- 多任务（multi task）在一个系统中可以同时运行多个程序，即有多个独立运行的任务，每个任务对应一个进程。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程

- **线程：**比进程更小的执行单位，一个进程中可以包含多个线程。
  - 一个进程在执行过程中，为了同时完成多项操作，可以产生多个线程，形成多条执行线索。
  - 每个线程都有它自身的产生、存在和消亡的过程。
- 一个进程中的所有线程都在该进程的虚拟地址空间中，**使用该进程的全局变量和系统资源**。
- 操作系统给每个线程分配不同的CPU时间片，**在某一时刻，CPU只执行一个时间片内的线程，多个时间片中的相应线程在CPU内轮流执行**。

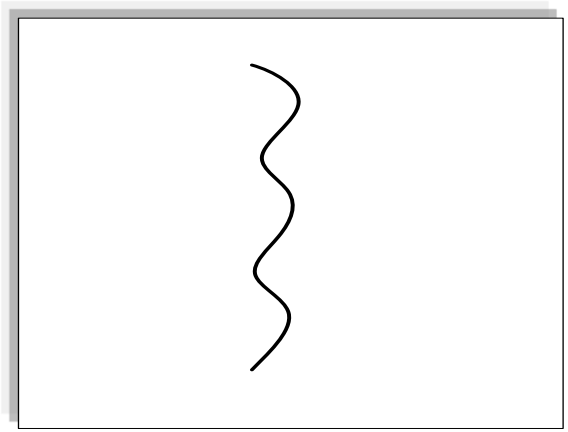
北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程和进程之间的关系

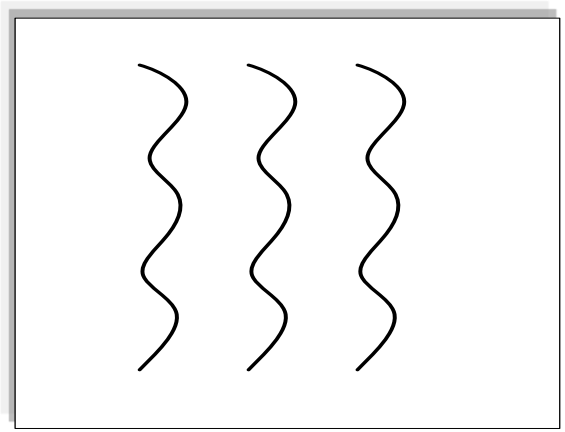
- 多进程环境中每一个进程既包括其所要执行的指令，也包括执行指令所需的任何系统资源，如CPU、内存空间、I/O端口等，不同进程所占用的系统资源相对独立；
- 线程是比进程单位更小的执行单位，多线程环境中每一个线程都隶属于某一进程，由进程触发执行，在系统资源的使用上，属于同一进程的所有线程共享该进程的系统资源；
  - 与进程不同的是线程本身即没有入口，也没有出口，其自身也不能独立运行，它栖身于某个进程之中，由进程启动运行，完成其任务后，自动终止，也可以由进程使之强制终止。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

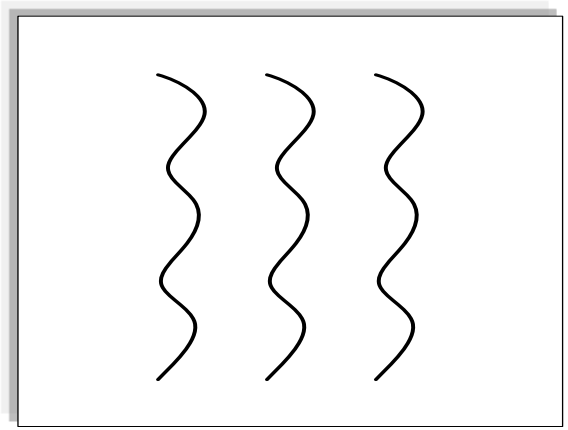
进程、线程示意图

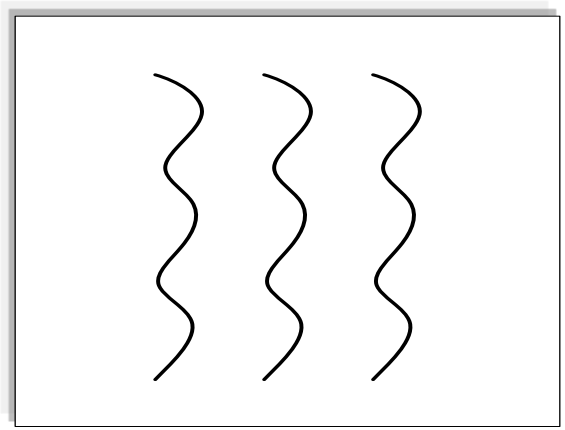


单进程　单线程



单进程　多线程





多进程　多线程

多线程程序设计

- 是指单个程序包含并发执行的多个线程。当多线程程序执行时，该程序对应的进程中就有多个控制流在同时执行，即具有并发执行的多个线程；
- 例如：
  - 影院售票问题
  - PV操作问题
  - Web Server接受客户端的请求问题
  - 银行问题
  - 网络聊天(一对多)程序

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

PV操作

- 多位生产者生产产品给消费者消费，生产者将生产的产品放到产品线，多位消费者从产品线取走产品消费。
- 产品线上最多容纳100个产品；
- 当产品线上有空位时生产者才可以生产，否则要等待，同时给消费者发送消费的信号；
- 当产品线上有产品时消费者才可以消费，否则要等待，同时给生产者发送生产的信号；
- 产品线上同一时间只能有一个生产者或消费者；

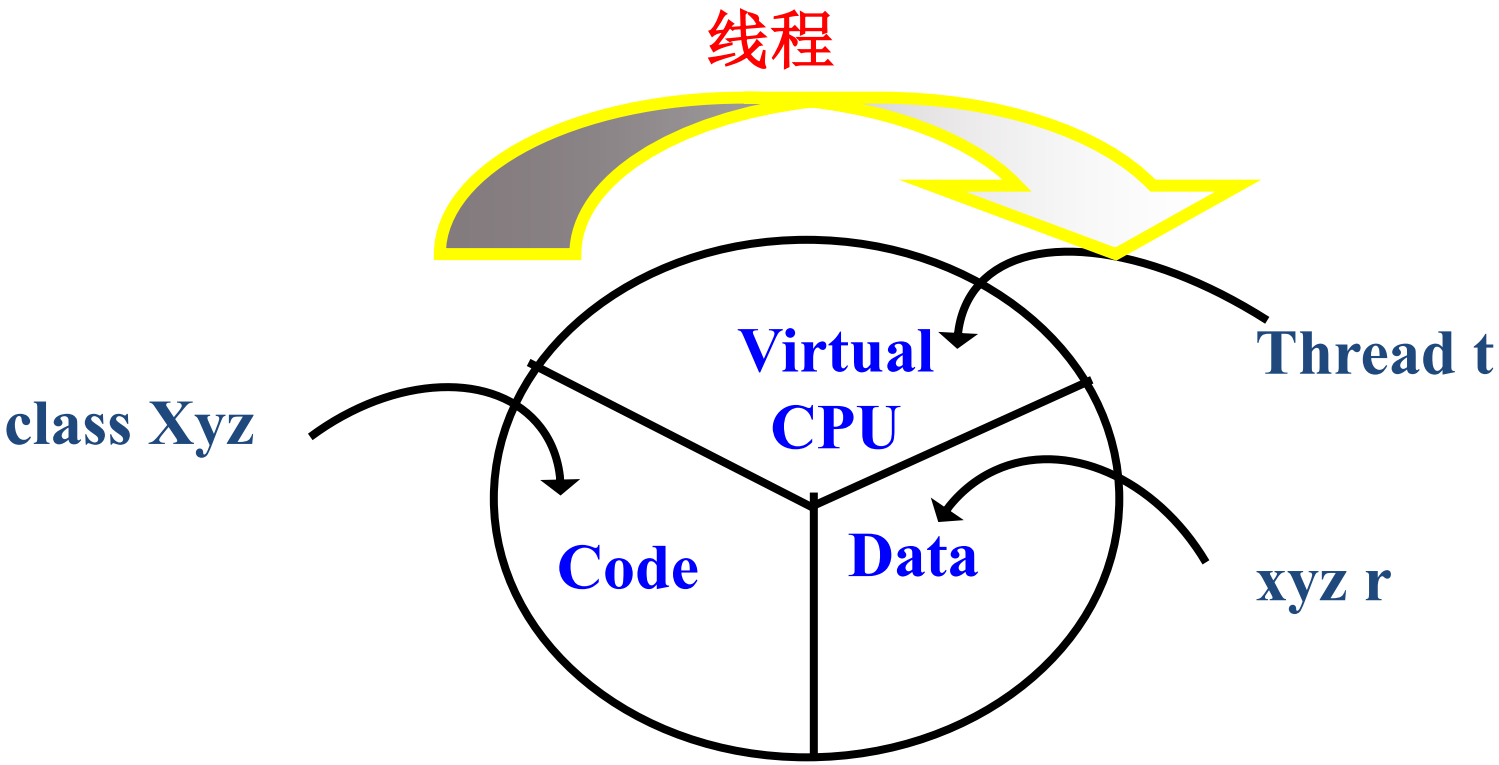
北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

为什么要使用多线程？（系统开销）

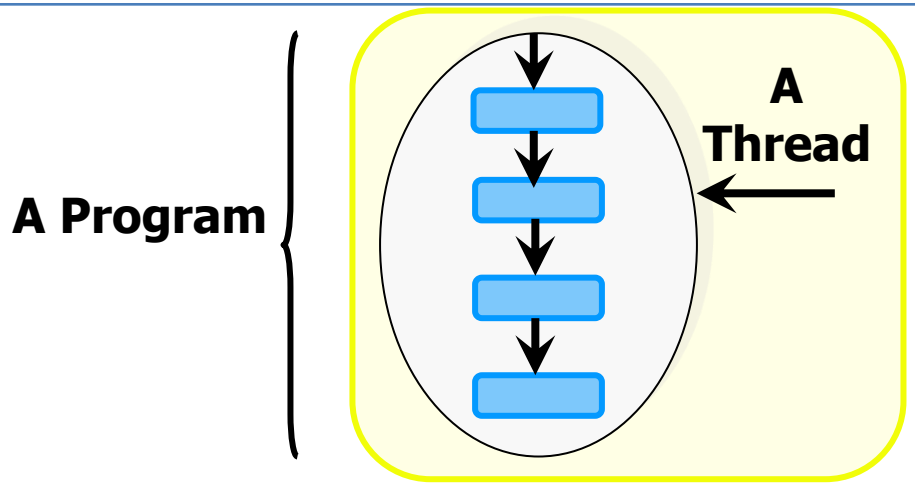
- 线程之间共享相同的内存单元(代码和数据)，因此在线程间切换，不需要很大的系统开销，所以线程之间的切换速度远远比进程之间快，线程之间的通信也比进程通信快的多。
- 多个线程轮流抢占CPU资源而运行时，从微观上讲，一个时间里只能有一个作业被执行，在宏观上可使多个作业被同时执行，即等同于要让多台计算机同时工作，使系统资源特别是CPU的利用率得到提高，从而可以提高整个程序的执行效率。

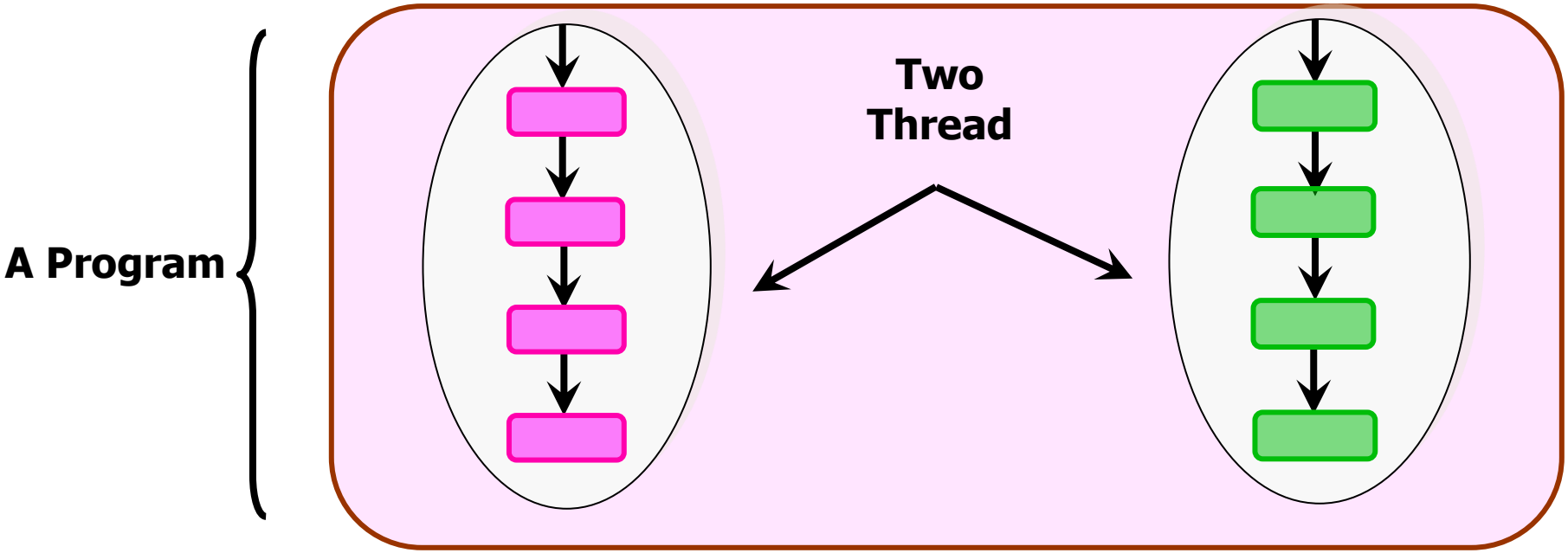
北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程运行环境



程序、进程和线程示意图





小结

1. 不同进程所占用的系统资源相对独立。每一个进程既包括其所要执行的指令, 也包括了执行指令所需的任何系统资源, 如CPU、内存空间、I/O端口等。
2. 在系统资源的使用上，隶属于同一个进程的多个线程共享上述资源。
3. 多线程环境中每一个线程都隶属于某一进程,由进程触发执行；
4. 线程之间切换的速度比进程切换要快得多（隶属于同一进程的线程）。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Java线程的运行机制

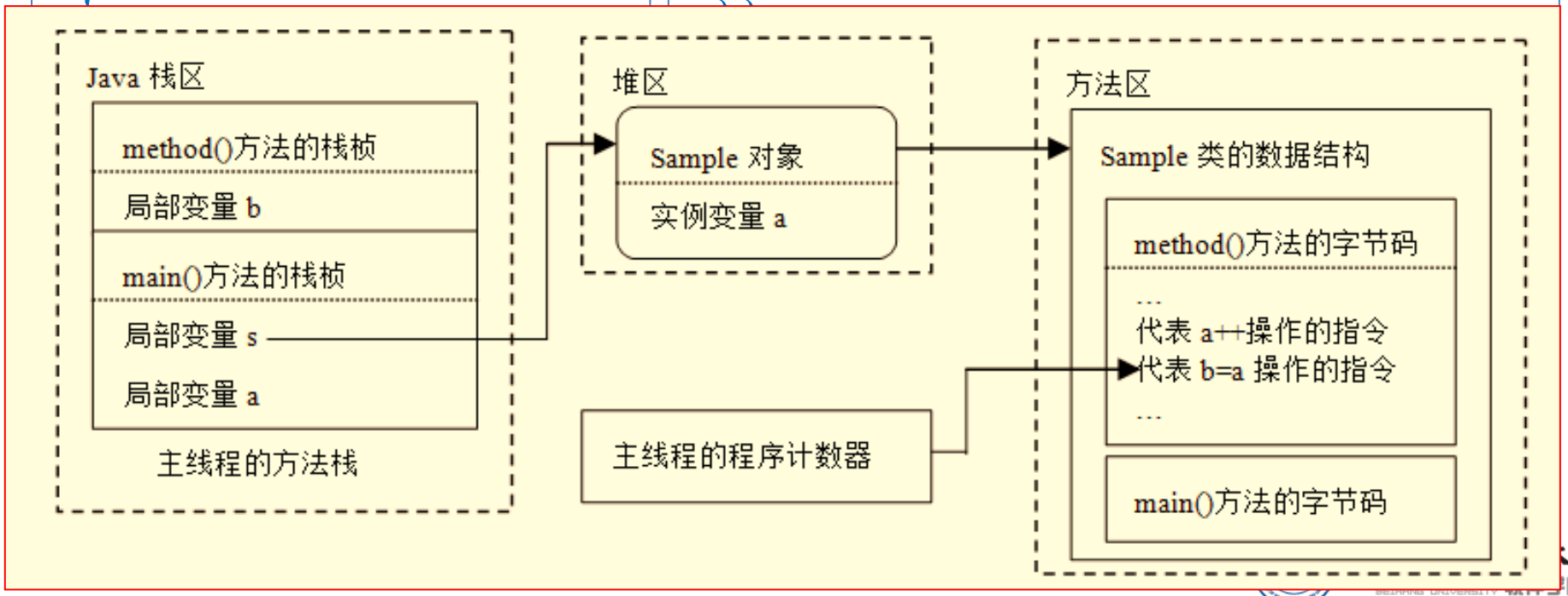
- 每个线程都有一个独立的程序计数器和方法调用栈（method invocation stack）：
  - 程序计数器：程序计数器（Program Counter Register）是一块较小的内存空间，它可以看做是当前线程所执行的字节码的行号指示器。在虚拟机的概念模型里（仅是概念模型，各种虚拟机可能会通过一些更高效的方式去实现），字节码解释器工作时就是通过改变这个计数器的值来选取下一条需要执行的字节码指令，分支、循环、跳转、异常处理、线程恢复等基础功能都需要依赖这个计数器来完成。
  - 方法调用栈：简称方法栈，用来跟踪线程运行中一系列的方法调用过程，栈中的元素称为栈桢。每当线程调用一个方法，就会向方法栈压入一个新桢，桢用来存储方法的参数、局部变量和运算过程中的临时数据。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

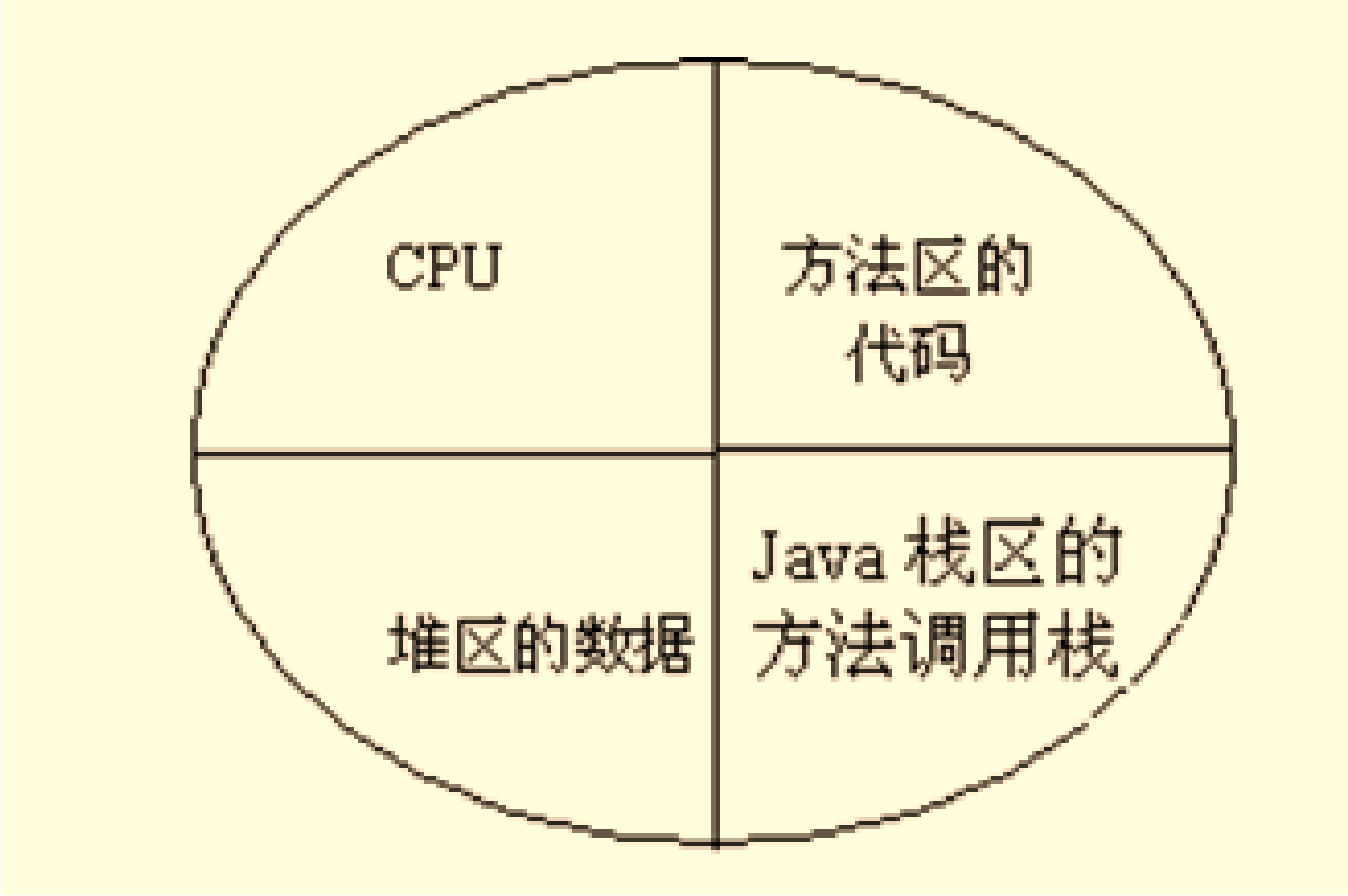
线程的栈帧

```
public class Sample{
  private int a;  //实例变量
  public int method(){
    int b=0;  //局部变量
    a++;
    b=a;
    return b;
  }
```

```
public static void main(String args[]){
    Sample s=null;  //局部变量
    int a=0;  //局部变量
    s=new Sample();
    a=s.method();
    System.out.println(a);
  }}
```



线程运行中需要的资源



主要内容

- 进程的概念
- 线程的概念
- **线程的调度**
- 创建和启动线程
  - Thread线程类、
  - Runnable接口
- 多个线程的同步
- 线程之间的通信

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程的调度（1）

- 微观上，在一台只具有一个CPU的机器上，CPU在同一时间只能分配给一个线程做一件事。
- 当有多于一个的线程工作时，在Java中，线程调度通常是*抢占式(即哪一个线程先抢到CPU资源则先运行)*，而不是分时间片式。
- 一旦一个线程获得执行权，这个线程将持续运行下去，直到它运行结束或因为某种原因而阻塞，或者有另一个高优先级线程就绪（这种情况称为低优先级线程被高优先级线程所抢占）。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程的调度（2）

- 所有被阻塞的线程按次序排列，组成一个阻塞队列。例如：
  - 因为需要等待一个较慢的外部设备，例如磁盘或用户。
  - 让处于运行状态的线程调用Thread.sleep()方法。
  - 让处于运行状态的线程调用另一个线程的join()方法。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程的调度（3）

- 所有就绪但没有运行的线程则根据其优先级排入一个就绪队列。
- 当CPU空闲时，如果就绪队列不空，就绪队列中第一个具有最高优先级的线程将运行。
- 当一个线程被抢占而停止运行时，它的运行态被改变并放到就绪队列的队尾；
- 一个被阻塞（可能因为睡眠或等待I/O设备）的线程就绪后通常也放到就绪队列的队尾。

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程的调度与优先级（1）

- 线程的调度是按：
  1. 其优先级的高低顺序执行的；
  2. 同样优先级的线程遵循“先到先执行的原则”；

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程的调度与优先级（1）

- 线程优先级：范围 1~10 （10 级）。数值越大，级别越高
  - Thread 类定义的 3 个常数：
    - MIN_PRIORITY 最低(小)优先级（值为1）
    - MAX_PRIORITY 最高(大)优先级（值为10）
    - NORM_PRIORITY 默认优先级（值为5）
  - 线程创建时，继承父线程的优先级。
  - 常用方法：
    - getPriority( )：获得线程的优先级
    - setPriority( )：设置线程的优先级

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

ThreadPri.java

```
public class ThreadPri
{
    public static void main(String args[ ])
    {
        MyThread th[ ]=new MyThread[3];
        for(int i=0;i<3;i++){
                th[i]=new MyThread( );
                th[i].start( );
        }
        th[0].setPriority(Thread.MAX_PRIORITY);
        //th[1].setPriority(Thread.NORM_PRIORITY);
        th[2].setPriority(Thread.MIN_PRIORITY);
    }
}
class MyThread extends Thread
{
    public void run( )
    {
        for(int i=0;i<2;i++)
            System.out.println(getName( )+"线程的优先级是：\t"+getPriority( )+"。");
    }
}
```

```
Thread-1线程的优先级是:        10。
Thread-1线程的优先级是:        10。
Thread-2线程的优先级是:        5。
Thread-2线程的优先级是:        5。
Thread-3线程的优先级是:        1。
Thread-3线程的优先级是:        1。
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

主线程

- main( ) 方法——Application 应用程序
  - 每当用java命令启动一个Java虚拟机进程（ Application 应用程序），Java虚拟机就会创建一个主线程，该线程从程序入口main()方法开始执行。
- 当在主线程中创建 Thread 类或其子类对象时，就创建了一个线程对象。主线程就是上述创建线程的父线程。
- Programmer可以控制线程的启动、挂起与终止。

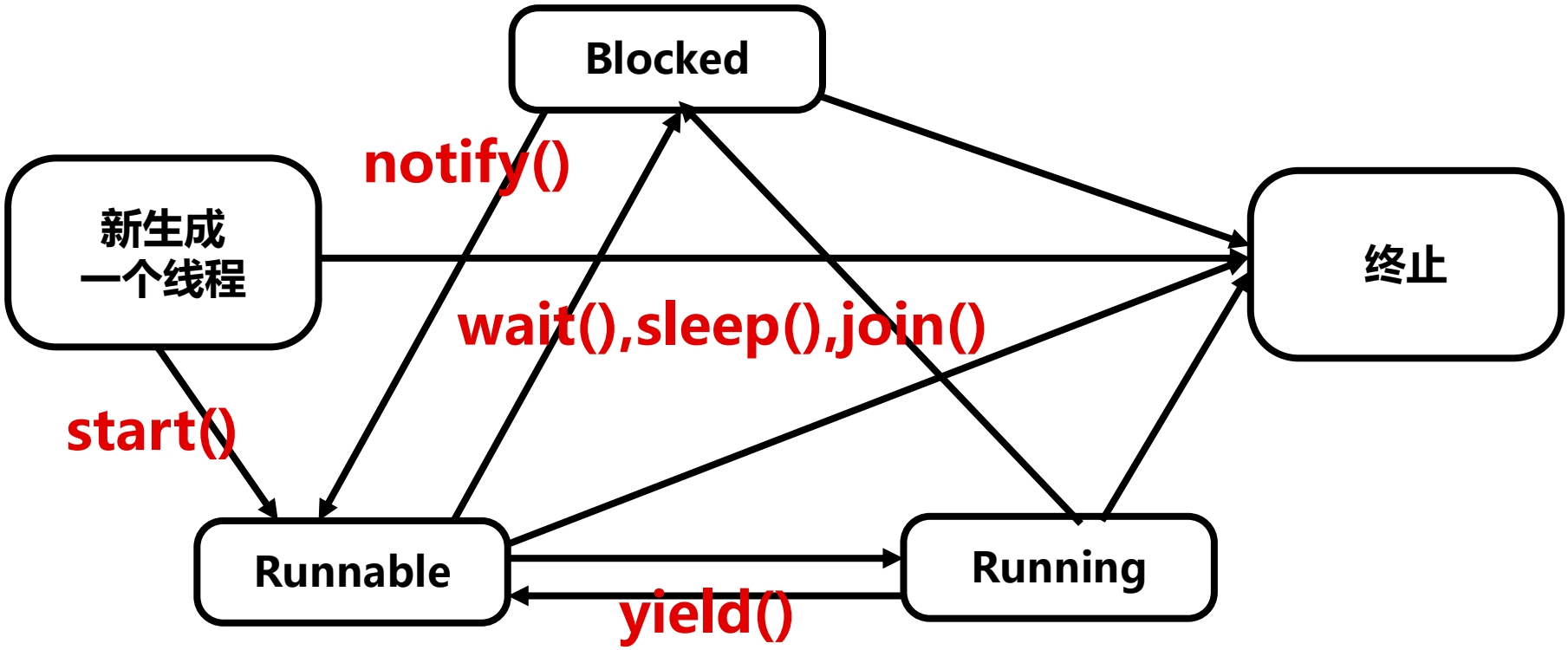
北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程的状态

- 在一个多线程运行环境中运行的线程可以有多种状态，也就是说一个线程在不同时刻会处于下列状态之一：
  - 新建、就绪、运行、阻塞、终止；

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程的状态


Blocked
notify()
新生成
一个线程
终止
wait(),sleep(),join()
start()
Runnable
Running
yield()

线程生命周期中的5种状态(1)

- 新建：当一个 Thread 类或其子类对象被创建时，新产生的线程处于新建状态，此时它已经有了相应的内存空间和其他资源。如：　Thread myThread=new Thread( );
- 就绪：调用 start( ) 方法来启动处于新建状态的线程后，将进入线程队列排队等待 CPU 服务，此时它已经具备了运行的条件，一旦轮到它来享用 CPU 资源时，就可以脱离创建它的主线程，开始自己的生命周期。

线程生命周期中的5种状态(2)

- 运行：当就绪状态的线程被调度并获得处理器资源时，便进入运行状态。
- 每一个 Thread 类及其子类的对象都有一个重要的 run( ) 方法，当线程对象被调用执行时，它将自动调用本对象的 run( ) 方法，从第一句开始顺序执行。
- run( ) 方法定义了这个线程的操作和功能。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程生命周期中的5种状态(3)

- 阻塞：一个正在执行的线程暂停自己的执行而进入的状态。引起线程由运行状态进入阻塞状态的可能情况：
  - 该线程正在等待 I/O 操作的完成
  - 网络操作
  - 为了获取锁而进入阻塞操作
  - 调用了该线程的 sleep( ) 方法
  - 调用了 wait( ) 方法
  - 让处于运行状态的线程调用另一个线程的join()方法

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程生命周期中的5种状态(4)

- 对应于不同进入阻塞状态的情况，采取不同的方法使其回到就绪状态：
  - I/O 操作：等待这个 I/O 操作完成后；
  - sleep( ) 方法：等待其指定的休眠事件结束后，自动脱离阻塞状态；
  - 调用 wait( ) 方法：调用 notify( ) 或 notifyAll( ) 方法；

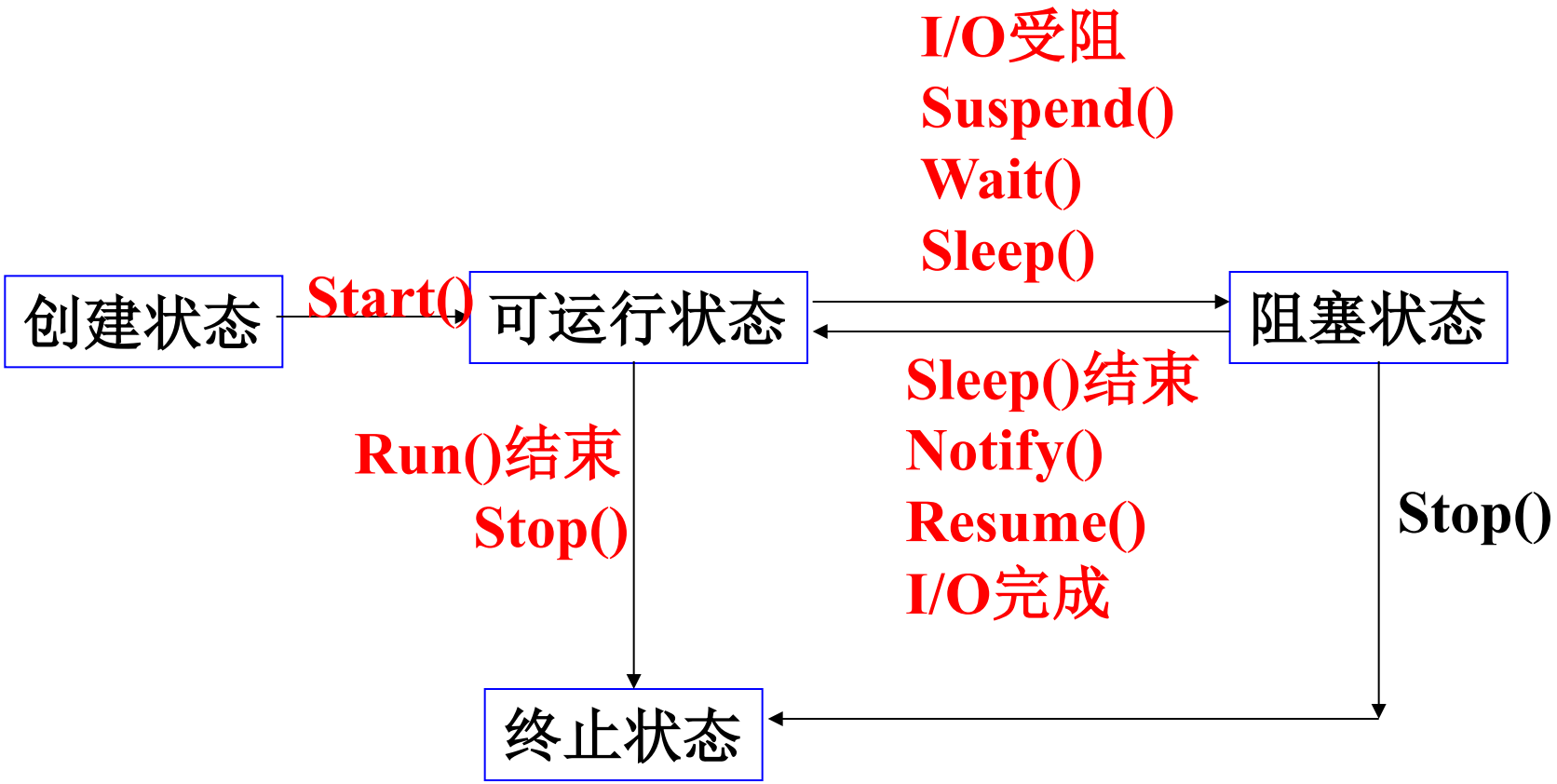
北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程生命周期中的5种状态(5)

- 终止：
  - 自然终止：线程完成了自己的全部工作
  - 强制终止：在线程执行完之前，调用 stop( ) 或 destroy( ) 方法终止线程

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

线程的状态及转换


I/O受阻
Suspend()
Wait()
Sleep()
创建状态
Start()
可运行状态
阻塞状态
Sleep()结束
Notify()
Resume()
I/O完成
Run()结束
Stop()
Stop()
终止状态

主要内容

- 进程的概念
- 线程的概念
- 线程的调度
- 创建和启动线程
  - Thread线程类、
  - Runnable接口
  - Callable接口
- 多个线程的同步
- 线程之间的通信

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

创建一个新的线程

- Java中的多线程是建立在Thread类, Runnable接口的基础上的，通常有两种办法让我们来创建一个新的线程：
  - 创建一个Thread类, 或者一个Thread子类的对象；
  - 创建一个实现Runnable接口的类的对象；

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Thread类介绍

Thread的构造方法
Thread();
Thread(Runnable target);
Thread(Runnable target, String name);
Thread(String name);
Thread(ThreadGroup group, Runnable target);
Thread(ThreadGroup group, Runnable target, String name);
Thread(ThreadGroup group, Runnable target, String name, long stackSize);
Thread(ThreadGroup group, String name);

name:线程的名称,target:要运行的线程对象,group:线程组的名称,stackSize:堆栈的大小

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Thread构造方法

- Thread的构造方法一共有八个，这里根据命名方式分类，使用默认命名的构造方法如下
  - Thread()
  - Thread(Runnable target)
  - Thread(ThreadGroup group,Runnable target)
- 命名线程的构造方法如下：
  - Thread(String name)
  - Thread(Runnable target,Strintg name)
  - Thread(ThreadGroup group,String name)
  - Thread(ThreadGroup group,Runnable target,String name)
  - Thread(ThreadGroup group,Runnable target,String name,long stackSize)

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Thread构造方法

- 一个线程的创建肯定是由另一个线程完成的；
- 被创建线程的父线程是创建它的线程；
- 而main线程由JVM创建，而main线程又可以成为其他线程的父线程；
- 如果一个线程创建的时候没有指定ThreadGroup，那么将会和父线程同一个ThreadGroup。main线程所在的ThreadGroup称为main；

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

关于stackSize（了解）

- Thread构造方法中有一个stackSize参数，该参数指定了JVM分配线程栈的地址空间的字节数，对平台依赖性较高，在一些平台上：
  - 设置较大的值：可以使得线程内调用递归深度增加，降低StackOverflowError出现的概率
  - 设置较小的值：可以使得创建的线程数增多，可以推迟OutOfMemoryError出现的时间
  - 但是，在一些平台上该参数不会起任何作用。另外，如果设置为0也不会起到任何作用。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Thread类介绍

Thread的主要方法(共34个,其中5个不再使用了)	
static Thread currentThread();	int getPriority();
String getName();	ThreadGroup getThreadGroup();
void interrupt();	boolean isInterrupted();
void join(); join(long millis);	join(long millis, int nanos);
static void yield();	void setName(String name);
void setPriority(int priority);	void start();
boolean isAlive();	static void sleep(long millis, int nanos);
void run();	static void sleep(long millis);

countStackFrames(); suspend(); resume();
stop(); stop(Throwable obj);

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Thread类的常用方法(1)

1. currentThread()：返回当前运行的Thread对象。
2. setName()：设置线程的名字。
3. getName()：返回该线程的名字。
4. setPriority()：设置线程优先级。
5. getPriority()：返回线程优先级。
6. start()：启动一个线程。
7. run()：线程体,由start()方法调用,当run()方法返回时,当前的线程结束。
8. stop()：使调用它的线程立即停止执行。
9. isAlive()：如果线程已被启动并且未被终止，那么isAlive()：返回true。如果返回false，则该线程是新创建或是已被终止的。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Thread类的常用方法(1)

10. sleep(int n)：使线程睡眠n毫秒,n毫秒后,线程可以再次运行。
11. yield()：将CPU控制权主动移交到下一个可运行线程。
12. join()：方法join()将引起现行线程等待，直至方法join所调用的线程结束。
13. suspend()： 使线程挂起,暂停运行。
14. resume() 恢复挂起的线程,使其处于可运行状态（Runnable）。
15. wait()：
16. notify()：
17. notifyAll()：

suspend();
resume();
Stop();
目前已经不再使用。为什么？请看API.

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

注意事项

- 因为Java线程的调度不是分时的，所以你必须确保你的代码中的线程会不时地给另外一个线程运行的机会。
- 有三种方法可以做到一点：

①让处于运行状态的线程调用Thread.sleep()方法。

②让处于运行状态的线程调用Thread.yield()方法。

③让处于运行状态的线程调用另一个线程的join()方法。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

sleep()和yield()的区别（1）

① 这两个方法都是静态的实例方法。

② sleep()使线程转入阻塞状态，而yield()使线程转入runnable状态。

③ yield()给相同优先级或更高的线程运行机会，如果当前没有存在相同优先级的线程，则yield()什么都不做。而sleep()不会考虑线程的优先级，会给其他线程运行的机会，因此也会给相同或更低优先级线程运行机会。

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

sleep()和yield()的区别（2）

④ sleep()会有中断异常抛出，而yiled()不抛出任何异常。

⑤ sleep()方法具有更好的可移植性，因为yield()的实现还取决于底层的操作系统对线程的调度策略。

⑥ 对于yield()的主要用途是在测试阶段，人为的提高程序的并发性能，以帮助发现一些隐藏的并发错误，当程序正常运行时，则不能依靠yield方法提高程序的并发行能。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

方法wait()与sleep()方法的对比（1）

- 相同点
  - 方法wait()与sleep() 方法一样，都能使线程等待而停止运行

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

方法wait()与sleep()方法的对比（2）

- 其区别在于：
  - sleep()方法不会释放对象的锁，而wait()方法进入等待时，可以释放对象的锁，因而别的线程能对这些加锁的对象进行操作。
  - 所以，wait,notify和notifyAll都是与同步相关联的方法,只有在synchronized方法中才可以用。在不同步的方法或代码中则使用sleep()方法使线程暂时停止运行。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

sleep()方法

- sleep()有两个重载方法：
  - sleep(long mills)
  - sleep(long mills,int nanos)
- 但是在JDK1.5后，引入了TimeUnit，其中对sleep()方法提供了很好的封装，建议使用TimeUnit.XXXX.sleep()去代替Thread.sleep()：
  - TimeUnit.SECONDS.sleep(1);
    TimeUnit.MINUTES.sleep(3);

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

join() 方法:

- 作用：使当前正在运行的线程暂停下来，等待指定的时间后或等待调用该方法的线程结束后，再恢复运行

1. public final void join()throws InterruptedException;
2. public final void join(long millis,int nanos)throws InterruptedException;
3. public final void join(long millis)throws InterruptedException;

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

	Deprecated. *This method has been deprecated, as it is inherently deadlock-prone. If the target thread holds a lock on the monitor protecting a critical system resource when it is suspended, no thread can access this resource until the target thread is resumed. If the thread that would resume the target thread attempts to lock this monitor prior to calling resume, deadlock results. Such deadlocks typically manifest themselves as "frozen" processes. For more information, see Why are Thread.stop, Thread.suspend and Thread.resume Deprecated?.*
String	**toString**() Returns a string representation of this thread, including the thread's name, priority, and thread group.
static void	**yield**() Causes the currently executing thread object to temporarily pause and allow other threads to execute.

Methods inherited from class java.lang.Object
clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

应用线程类Thread创建线程

1. 将一个类定义为Thread的子类,那么这个类就可以用来创建线程。
2. 这个类中有一个至关重要的方法——public void run，这个方法称为线程体，它是整个线程的*核心*，线程所要完成任务的代码都定义在*线程体*中，实际上不同功能的线程之间的区别就在于它们线程体的不同

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

应用线程类创建线程

创建线程对象 → 线程子类名　线程对象名 =new 线程子类名（实际参数）

↓

启动一个线程 → 线程对象名.start();

↓

*线程体*的执行 → Run()方法的执行

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
import java.util.*;
class TimePrinter extends Thread {
    int pauseTime;
    String name;
    public TimePrinter(int x, String n) {
        pauseTime = x;
        name = n;
    }
    public void run() {
        while(true) {
            try {
        System.out.println(name + ":" + new Date(System.currentTimeMillis()))
                Thread.sleep(pauseTime);
            }
            catch(Exception e)
            {
                System.out.println(e);
            }
        }
    }
```

Thread.sleep()方法的使用，注意异常的抛出

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
static public void main(String args[]) {
    TimePrinter tp1 = new TimePrinter(1000, "Fast Guy");
    tp1.start();
    TimePrinter tp2 = new TimePrinter(3000, "Slow Guy");
    tp2.start();
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

运行 结果

```
Fast Guy:Mon May 31 15:29:21 CST 2021
Slow Guy:Mon May 31 15:29:21 CST 2021
Fast Guy:Mon May 31 15:29:22 CST 2021
Fast Guy:Mon May 31 15:29:23 CST 2021
Slow Guy:Mon May 31 15:29:24 CST 2021
Fast Guy:Mon May 31 15:29:24 CST 2021
Fast Guy:Mon May 31 15:29:25 CST 2021
Fast Guy:Mon May 31 15:29:26 CST 2021
Slow Guy:Mon May 31 15:29:27 CST 2021
Fast Guy:Mon May 31 15:29:27 CST 2021
Fast Guy:Mon May 31 15:29:28 CST 2021
Fast Guy:Mon May 31 15:29:29 CST 2021
Slow Guy:Mon May 31 15:29:30 CST 2021
Fast Guy:Mon May 31 15:29:30 CST 2021
Fast Guy:Mon May 31 15:29:31 CST 2021
Fast Guy:Mon May 31 15:29:32 CST 2021
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
import java.util.*;
class TimePrinter3 extends Thread {
    int pauseTime;
    String name;
    public TimePrinter3(int x, String n) {
        pauseTime = x;
        name = n;
    }
    public void run() {
        while(true) {
        System.out.println(name + ":" + new Date(System.currentTimeMillis()))
        yield();
        }
    }
    static public void main(String args[]) {
        TimePrinter tp1 = new TimePrinter(1000, "Fast Guy");
        tp1.start();
        TimePrinter tp2 = new TimePrinter(3000, "Slow Guy");
        tp2.start();
    }
}
```

Thead.yield()
方法

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
public class TestSubThread3{
    public static void main(String args[]){
        SubThread t1=new SubThread("线程_1");
        SubThread t2=new SubThread("线程_2");
        t1.start();
        t2.start();
        System.out.println("main()方法运行完毕！");
    }
}
class SubThread extends Thread{
    String str;
    public SubThread(String s){
        str=s;
    }
    public void run(){
        for(int i=1;i<=3;i++){
            for(long j=1;j<100000000;j++)
                ;
            System.out.println(str+": 第"+i+"次运行");}
        System.out.println(str+": 结束！");
    }
}
```

主线程和其他线程共存

运行结果

```
main()方法运行完毕!
线程_2:  第1次运行
线程_1:  第1次运行
线程_2:  第2次运行
线程_1:  第2次运行
线程_1:  第3次运行
线程_1:  结束!
线程_2:  第3次运行
线程_2:  结束!
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
public class TestSubThread1{
        public static void main(String[] args)throws InterruptedException{
                SubThread t1=new SubThread("First 线程");
                SubThread t2=new SubThread("Second 线程");
                t1.start();
                t2.start();
                Thread.sleep(5000);
                System.out.println("main()方法运行完毕！");
        }
}
class SubThread extends Thread{
        String str;
        public SubThread(String s){
                str=s;
        }
        public void run(){
                for(int i=1;i<=3;i++){
                        for(long j=1;j<100000000;j++)
                                ;
                        System.out.println(str+": 第"+i+"次运行");
                }
                System.out.println(str+": 结束！");
        }
}
```

Thread.sleep()方法的使用，注意异常的抛出

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

运行结果

```
First 线程: 第1次运行
Second 线程: 第1次运行
main()方法运行完毕!
First 线程: 第2次运行
Second 线程: 第2次运行
First 线程: 第3次运行
First 线程: 结束!
Second 线程: 第3次运行
Second 线程: 结束!
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
public class TestSubThread{
    public static void main(String args[]){
        SubThread t1=new SubThread("线程_1");
        SubThread t2=new SubThread("线程_2");
        t1.start();
        t2.start();
        try{
            t1.join();
            t2.join();
        }
        catch(InterruptedException e){
            System.out.println(e.toString());
        }
        System.out.println("main()方法运行完毕！");
    }
}
```

join()方法的使用，注意异常的抛出。保证了主线程最后结束

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

```java
class SubThread extends Thread{
    String str;
    public SubThread(String s){
        str=s;
    }
    public void run(){
        for(int i=1;i<=3;i++){
            for(long j=1;j<100000000;j++)
                ;
            System.out.println(str+": 第"+i+"次运行");
        }
        System.out.println(str+": 结束！");
    }
}
```

```
线程_2: 第1次运行
线程_1: 第1次运行
线程_2: 第2次运行
线程_1: 第2次运行
线程_2: 第3次运行
线程_2: 结束!
线程_1: 第3次运行
线程_1: 结束!
main()方法运行完毕!
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
public class TestMyThread{
    public static void main(String args[]){
        MyThread t=new MyThread(5);
        System.out.println("线程名:"+t.getName());//得到线程的名字
        //线程t没有启动，t.isAlive()得值为false
        System.out.println("线程启动?    "+t.isAlive());
        t.start();//线程t已启动，t.isAlive()得值为true
        System.out.println("线程启动?    "+t.isAlive());
        System.out.println("线程是否中断?"+t.isInterrupted());
        t.interrupt();
        System.out.println("线程是否中断?"+t.isInterrupted());
    }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

```java
class MyThread extends Thread{
    MyThread(int priority){
        setPriority(priority);
        setName("MyThread-"+priority);
    }
    public void run(){
        System.out.println(getName()+"运行");
        try{
            sleep(1000);
        }
        catch(InterruptedException e){
            System.out.println("线程休眠时调用interrupt()方法会抛出异常");
        }
        System.out.println(getName()+"结束!");
    }
}
```

```
线程名:MyThread-5
线程启动?        false
线程启动?        true
线程是否中断?false
线程是否中断?true
MyThread-5运行
线程休眠时调用interrupt()方法会抛出异常
MyThread-5结束!
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

应用Runnable接口创建线程

1. Runnable是Java中用以实现线程的接口，从根本上讲，任何实现线程功能的类都必须实现该接口。
   - Thread(Runnable target);
   - Thread(Runnable target, String name);
2. Runnable接口中只定义了一个方法就是run()方法，也就是线程体。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
import java.util.*;
class TimePrinter1 implements Runnable {
    int pauseTime;
    String name;
    public TimePrinter1(int x, String n) {
        pauseTime = x;
        name = n;
    }
    public void run() {
        while(true) {
            try {
                System.out.println(name + ":" + new
                Date(System.currentTimeMillis()));
                Thread.sleep(pauseTime);
            }
            catch(Exception e) {
                System.out.println(e);
            }
        }
    }
    static public void main(String args[]) {
        Thread t1 = new Thread(new TimePrinter1(1000, "Fast Guy"));
        t1.start();
        Thread t2 = new Thread(new TimePrinter1(3000, "Slow Guy"));
        t2.start();
    }
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
class MyThread implements Runnable{
    String str;
    int count;
    public MyThread(String s,int count){
        str=s;
        this.count=count;
    }
    public void run(){
        for(int i=1;i<=5;i++){
            for(long j=1;j<1000000000;j++)
                ;
            System.out.println(Thread.currentThread().getName()+": 第"+i+"次运行");
        }
        System.out.println(Thread.currentThread().getName()+": 结束！");
    }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

```java
public class TestMyThread{
    public static void main(String args[])throws InterruptedException{
        MyThread parent1=new MyThread("线程_1",100);
        Thread t1=new Thread(parent1,"t1");
        Thread t2=new Thread(parent1,"t2");

        MyThread parent2=new MyThread("线程_2",200);
        Thread t3=new Thread(parent2,"t3");
        Thread t4=new Thread(parent2,"t4");
        t1.start();
        t2.start();
        t3.start();
        t4.start();
        t1.join();
        t2.join();
        t3.join();
        t4.join();
        System.out.println(Thread.currentThread().getName()+"方法运行完毕！");
    }
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

运行结果

```
t1:  第1次运行
t4:  第1次运行
t3:  第1次运行
t2:  第1次运行
t1:  第2次运行
t4:  第2次运行
t3:  第2次运行
t2:  第2次运行
t1:  第3次运行
t4:  第3次运行
t2:  第3次运行
t3:  第3次运行
t1:  第4次运行
t4:  第4次运行
t3:  第4次运行
t2:  第4次运行
t1:  第5次运行
t1:  结束!
t3:  第5次运行
t3:  结束!
t2:  第5次运行
t2:  结束!
t4:  第5次运行
t4:  结束!
main方法运行完毕!
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

适用于采用实现Runnable接口方法的情况

① 因为Java只允许单继承，如果一个类已经继承了Thread，就不能再继承其他类。

② 特别是在除了run()方法以外,并不打算重写Thread类的其它方法的情况下,以实现Runnable接口的方式生成新线程就显得更加合理了。

③ 第三种使用场合见上例，涉及到数据共享的时候；

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

终止线程

- 当线程执行完run()方法，它将自然终止运行。
- Thread有一个stop()方法，可以强制结束线程，但这种方法是不安全的。因此，在stop()方法已经被废弃。
- 实际编程中，一般是定义一个标志变量，然后通过程序来改变标志变量的值，从而控制线程从run()方法中自然退出。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

示例代码

```
public class MyThreadStop extends Thread{
        int a; boolean flag=false;
        public void run(){
                while(!flag){

                System.out.println(a++);
                }
        }
        public void setFlag(boolean _flag){
                flag=_flag;
        }
        public static void main(String args[]){
                MyThreadStop t=new MyThreadStop();
                t.start();
                try{ Thread.sleep(1000);}catch(Exception e){}
                t.setFlag(true);
        }
}
```


北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

小结：创建用户多线程的步骤（1）

1.创建一个Thread类的子类
2.在子类中将希望该线程做的工作写到run()里面
3.生成该子类的一个对象
4.调用该对象的start()方法

```
class MyThread extends Thread{
    public void run(){... ...}
    //其它方法等
}
class MyClass{
    public static void main(String[] args){
        MyThread mt = new MyThread();
        mt.start();
    }
    //其它方法等
}
```

省略号代表的是我们想让这个线程完成的工作

调用start(),就会生成一个新的线程,并开始执行run()里规定的任务

小结：创建用户多线程的步骤(2)

1.创建一个实现Runnable接口的类
2.在该类中将希望该线程做的工作写到run()里面
3.生成该类的一个对象
4.用上述对象去生成Thread类的一个对象
5.调用Thread类的对象的start()方法

```
class MyRunnable implements Runnable{
    public void run(){… …}
    //其它方法等
}
class MyClass{
    public static void main(String[] args){
        MyRunnable mr = new MyRunnable();
        Thread t = new Thread(mr);
        t.start();
    }
    //其它方法等
}
```

省略号代表的是我们想让这个线程完成的工作

调用Thread对象的start(),就会生成一个新的线程,并开始执行MyRunnable类的run()里规定的任务

小结：创建用户多线程的步骤(3)

1.创建一个实现Runnable接口的类
2.在该类中将希望该线程做的工作写到run()里面
3.写一个start()方法,在里面创建Thread,调用start()
4.生成Runnable类的一个对象
5.调用该类对象的start()方法

```
class MyRunnable implements Runnable{
    public void run(){... ...}
    public void start(){
        new Thread(this).start();
    }
    //其它方法等
}
class MyClass{
    public static void main(String[] args){
        MyRunnable mr = new MyRunnable();
        mr.start();
    }
    //其它方法等
}
```

省略号代表的是我们想让这个线程完成的工作

以this为参数生成一个Thread类的对象,并调用它的start()方法

调用start(),就会生成一个新的线程,并开始执行run()里规定的任务

小结：

- 两种实现方式的区别和联系
  - 在程序开发中只要是多线程尽量以实现Runnable接口为主，因为实现Runnable接口相比继承Thread类有如下好处：
    - 1、避免单继承的局限，一个类可以实现多个接口。
    - 2、适合于资源的共享
- Runnable接口和Thread之间的联系
  - public class Thread extends Object implements Runnable

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Java中Runnable接口的局限性

1. Runnable接口的**run()**方法是**void**类型，**不能返回任何值**。
2. **Runnable**接口**不允许抛出任何已检查的异常**（编译时捕获的异常。例如：**Exception**）。

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Callable接口

```
public interface Runnable {
    public abstract void run();
}
```

- Java中实现多线程有三种方法：
  - 一种是继承Thread类；
  - 第二种是实现Runnable接口；
    - Runnable接口位于java.lang包
  - 第三种是实现Callable接口；
    - Callable接口位于java.util.concurrent包中。Callable接口的定义为：

```
@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception;
}
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

案例1：

```java
public class CallableThread implements Callable<String> {
    @Override
    public String call() throws Exception {
        return "线程名称: " + Thread.currentThread().getName();
    }
}
```

```java
CallableThread callableThread = new CallableThread();
FutureTask<String> futureTask = new FutureTask<>(callableThread);
Thread thread = new Thread(futureTask);
thread.start();
String result = futureTask.get();
System.out.println("执行结果= " + result);
```

使用 FutureTask.get()方法获取执行结果

借助FutureTask，因为 Thread类没有接受Callable的构造函数。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Java中实现Callable接口的步骤

1. **创建一个实现Callable接口的类。**
2. 重写类中的**Callable**接口的**call()方法。**
3. **通过传递实现了Callable**接口的类对象来创建**FutureTask**对象。
4. **通过传递创建的FutureTask**对象在主类中创建一个**Thread**对象。
5. 调用**Thread**对象的**start()**方法。
6. **使用FutureTask**对象的**get()**方法获取可调用类返回的结果。

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Callable接口的特点

- **Callable**接口的 call() 方法可以**返回**任何对象。
- **Callable**接口的 call() 方法可以**抛出**已检查的异常（编译时捕获的异常。例如：**Exception**）和未检查的异常（运行时发生的异常。例如：**RuntimeException**）。
- **可以使用FutureTask**对象的**cancel()**方法取消计算。

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

在日常的开发中，推荐使用线程池的方式来使用

```
CallableThread callableThread = new CallableThread();
ExecutorService executor = Executors.newSingleThreadExecutor();
Future<String> future = executor.submit(callableThread);
String result = future.get();
System.out.println("任务执行结果: " + result);
executor.shutdown();
```


北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

案例2:

```java
package com.buaa.test;
import java.util.ArrayList;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;

class TaskWithResult implements Callable<String>{
    private int id;
    private static int count =10;
    private final int time =count--;
    public TaskWithResult(int id){
        this.id = id;
    }

    @Override
    public String call() throws Exception {
        TimeUnit.MILLISECONDS.sleep(100);
        return "Result of TaskWithResult : "+ id+", Time= "+time;
    }

}
```

```java
public class CallableDemo {
    public static void main(String[] args)
            throws InterruptedException, ExecutionException {
        ExecutorService exec = Executors.newCachedThreadPool();
        ArrayList<Future<String>> results =new ArrayList<Future<String>>();
        for(int i=0;i<10;i++){
            results.add(exec.submit(new TaskWithResult(i)));
        }

        for(Future<String> fs : results){
            System.out.println(fs.get());
        }
    }

}
```

```
Result of TaskWithResult : 0, Time= 10
Result of TaskWithResult : 1, Time= 9
Result of TaskWithResult : 2, Time= 8
Result of TaskWithResult : 3, Time= 7
Result of TaskWithResult : 4, Time= 6
Result of TaskWithResult : 5, Time= 5
Result of TaskWithResult : 6, Time= 4
Result of TaskWithResult : 7, Time= 3
Result of TaskWithResult : 8, Time= 2
Result of TaskWithResult : 9, Time= 1
```

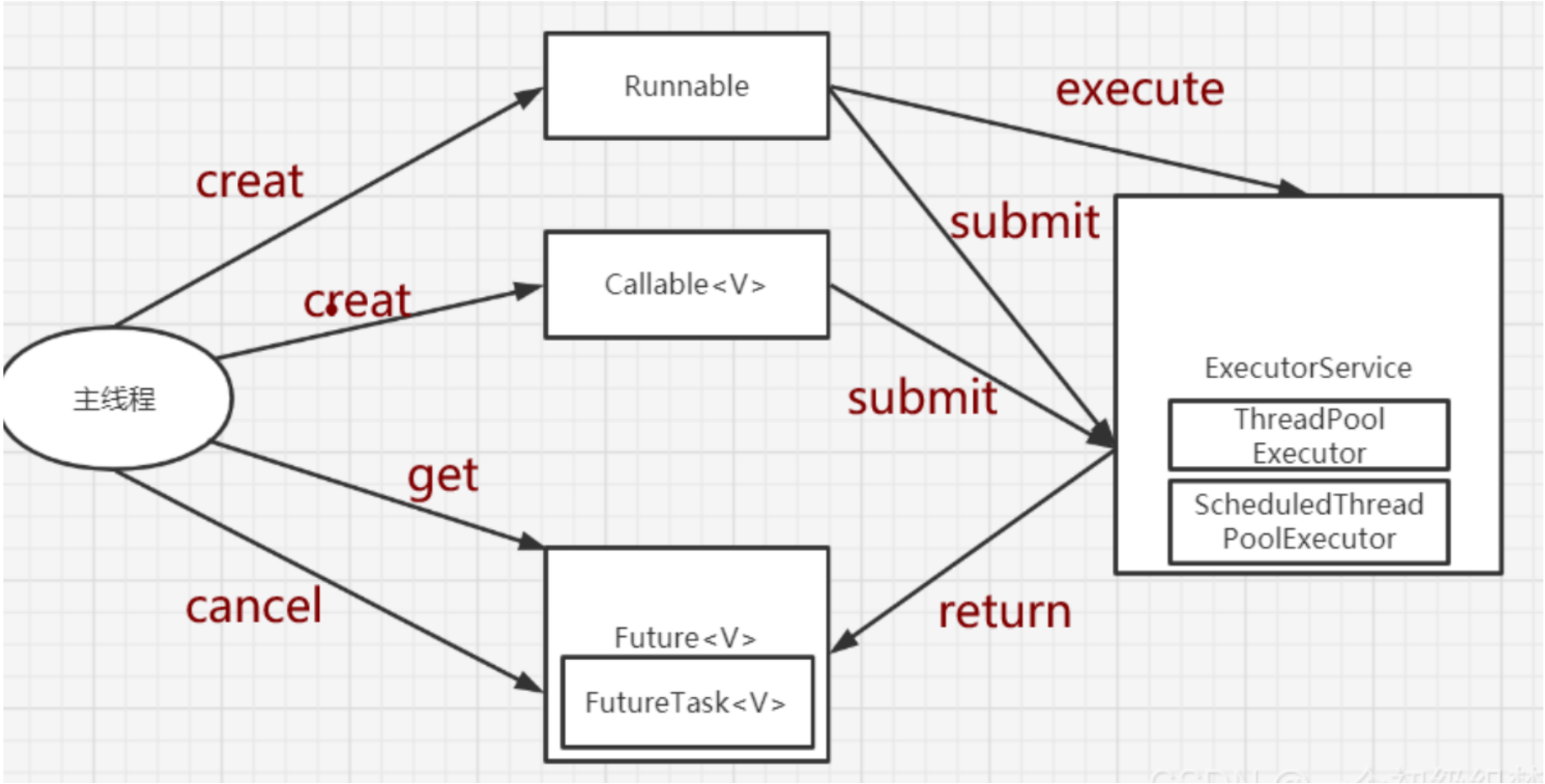

北京航空航天大學 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Callable 和 Runnable 的使用方法对比

- 1、Callable 使用 call（）方法， Runnable 使用 run() 方法
- 2.、Callable的任务执行后可返回值，而Runnable的任务不能有返回值(是void)
- 3、call() 可以抛出受检查的异常，而run()不能抛出受检查的异常。
- 4、运行Callable任务可以拿到一个Future对象，表示异步计算的结果。它提供了检查计算是否完成的方法，以等待计算的完成，并检索计算的结果。通过Future对象可以了解任务执行情况，可取消任务的执行，还可获取执行结果。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

Executor框架


Runnable
execute
creat
submit
Callable<V>
creat
主线程
submit
ExecutorService
ThreadPool Executor
ScheduledThread PoolExecutor
get
cancel
return
Future<V>
FutureTask<V>

execut和submit方法区别

- ExcutorService中的excutor和submit方法的都是将一个线程任务添加到线程池中并执行，区别在于：
  - 1、excutor没有返回值，submit有返回值，并且返回执行结果Future对象
  - 2、excutor不能提交Callable任务，只能提交Runnable任务，submit两者任务都可以提交
  - 3、在submit中提交Runnable任务，会返回执行结果Future对象，但是Future调用get方法将返回null（Runnable没有返回值）

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

实际工作中如何选择？

- 1、取舍的基本原则就是需不需要返回值，如果不需要返回值，那直接就选 Runnable。如果有返回值的话，使用Callable。
- 2、另外一点就是是否需要抛出异常， Runnable是不接受抛出异常的，Callable可以抛出异常。

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院

- 3、Runnable适合那种纯异步的处理逻辑。比如每天定时计算报表，将报表存储到数据库或者其他地方，只是要计算，不需要马上展示，展示内容是在其他的方法中单独获取的。（比如那些非核心的功能，当核心流程执行完毕后，非核心功能就自己去执行）
- 4、Callable适用于那些需要返回值或者需要抛出checked exception的情况，比如对某个任务的计算结果进行处理。在Java中，常常使用callable来实现异步任务的处理，以提高系统的吞吐量和响应速度

北京航空航天大学 COLLEGE OF SOFTWARE BEIHANG UNIVERSITY 软件学院